

Agent Phantom Recovery

09-backend-architecture.md

Production Backend Architecture

Version: 1.0

1. Purpose

This document defines the production-grade backend architecture for Agent Phantom Recovery.

The backend is designed for:

- Long-horizon execution
- Multi-agent orchestration
- Multi-LLM reasoning
- Tool execution
- Memory persistence
- Repository intelligence
- Verification-driven workflows
- Real-time workspace updates
- Antigravity IDE integration

This architecture prioritizes:

1. Correctness
2. Reliability
3. Traceability
4. Scalability
5. Completion

over raw response speed.

2. Backend Philosophy

Traditional AI systems follow:

```
User
↓
LLM
```

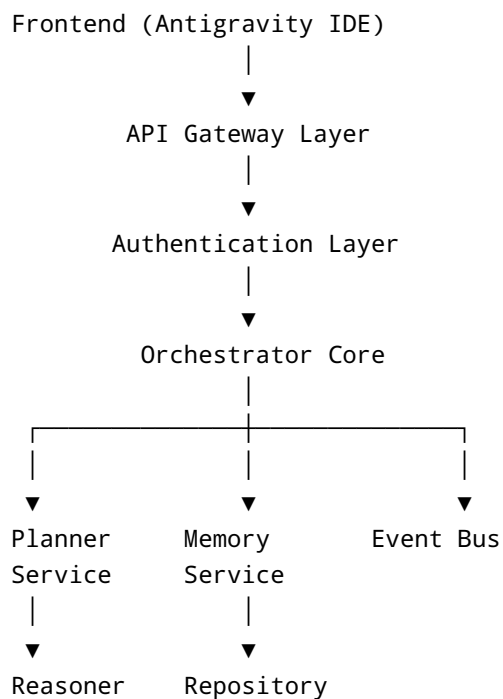
↓
Response

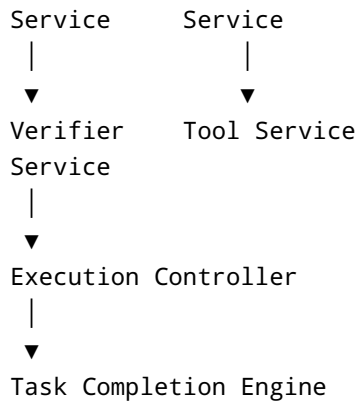
Agent Phantom Recovery follows:

```
User Goal
↓
Orchestrator
↓
Planner
↓
Evidence Collection
↓
Reasoner
↓
Verifier
↓
Tool Execution
↓
Memory Update
↓
Completion
```

The backend is therefore an execution platform rather than a chatbot backend.

3. High-Level Backend Architecture





4. Architectural Layers

The backend is divided into eight major layers.

Layer 1 → Interface Layer
Layer 2 → API Layer
Layer 3 → Orchestration Layer
Layer 4 → Intelligence Layer
Layer 5 → Memory Layer
Layer 6 → Tool Layer
Layer 7 → Execution Layer
Layer 8 → Storage Layer

Each layer has strict responsibilities.

5. Interface Layer

Purpose

Communicate with Antigravity IDE.

Responsibilities

- WebSocket updates
- Streaming responses
- Live task progress
- Browser synchronization
- Terminal synchronization
- Source code updates

Inputs

User Commands
Task Requests
Workspace Actions

Outputs

Execution State
Task Progress
Logs
Results
Verification Reports

6. API Gateway Layer

Purpose

Single entry point for all requests.

Responsibilities

- Authentication
 - Authorization
 - Routing
 - Rate limiting
 - Request validation
 - Request tracing
-

APIs

/auth/*
/tasks/*
/agents/*
/memory/*
/repos/*
/tools/*
/reports/*

Why API Gateway?

Without an API gateway:

- Services become tightly coupled
- Security becomes inconsistent
- Observability becomes difficult

The gateway centralizes platform control.

7. Authentication Service

Purpose

Identity management.

Responsibilities

- Login
 - Logout
 - Session creation
 - Token validation
 - Permission enforcement
-

Future Support

- OAuth
 - Enterprise SSO
 - Organization accounts
-

8. Orchestrator Core

Purpose

The orchestrator is the brain of the backend.

Responsibilities

- Task lifecycle management
- State transitions
- Agent coordination
- Workflow execution
- Retry handling

- Recovery handling
-

Important Rule

The orchestrator never performs reasoning.

It only manages execution.

Example

```
Task Created
↓
Plan Requested
↓
Planner Called
↓
Tool Required
↓
Tool Executed
↓
Result Observed
↓
Reasoner Called
↓
Verification Requested
↓
Complete
```

9. Task State Machine

Every task must have a state.

States

```
CREATED
PLANNING
COLLECTING_EVIDENCE
ANALYZING
EXECUTING
VERIFYING
COMPLETED
FAILED
```

BLOCKED
PAUSED

Benefits

- Easy recovery
 - Clear progress
 - Better debugging
 - Traceability
-

10. Planner Service

Purpose

Convert goals into executable plans.

Responsibilities

- Goal analysis
 - Task decomposition
 - Dependency analysis
 - Tool selection
 - Prioritization
-

Example

User:

Fix authentication bug

Planner:

1. Analyze repo
 2. Run tests
 3. Inspect auth module
 4. Generate hypothesis
 5. Patch code
 6. Verify
-

Output

```
{
  "task_id": "123",
  "plan": [],
  "dependencies": [],
  "success_criteria": []
}
```

11. Repository Intelligence Service

Purpose

Transform large repositories into manageable evidence.

Responsibilities

- Repository ingestion
 - Index generation
 - AST generation
 - Call graph creation
 - Dependency graph creation
 - Symbol extraction
-

Pipeline

```
Repository
↓
Indexer
↓
AST Parser
↓
Dependency Graph
↓
Call Graph
↓
Candidate Ranking
↓
Reasoner
```

Why This Matters

Reasoning over:

200,000 lines

is expensive.

Reasoning over:

200 important lines

is practical.

12. Memory Service

Purpose

Persistent contextual intelligence.

Memory Types

Working Memory
Session Memory
Project Memory
Experience Memory

Responsibilities

- Storage
 - Retrieval
 - Ranking
 - Summarization
 - Compression
-

Retrieval Pipeline

Goal
↓
Memory Query

↓
Rank Results
↓
Assemble Context
↓
Return

13. Reasoner Service

Purpose

Deep technical analysis.

Responsibilities

- Root cause analysis
- Architecture reasoning
- Patch generation
- Bug investigation
- Repository understanding

Inputs

Repository Evidence
Logs
Tests
Memory
Plan

Outputs

Findings
Patch Plans
Fixes
Explanations

Critical Requirement

Reasoner must not directly approve itself.

Verification is external.

14. Verifier Service

Purpose

Independent quality control.

Responsibilities

- Validate findings
 - Validate patches
 - Detect regressions
 - Challenge assumptions
-

Workflow

```
graph TD
    A[Reasoner Output] --> B[Verifier Review]
    B --> C[Accept  
or  
Reject]
```

Why Separate?

Using the same model for:

```
Reasoning
+
Verification
```

creates correlated failures.

Independent verification improves reliability.

15. Tool Service

Purpose

Unified access to tools.

Supported Tools

```
Terminal
Browser
Search
Filesystem
Code Runner
Git
GitHub
```

Responsibilities

- Registration
 - Discovery
 - Execution
 - Monitoring
-

Tool Contract

Every tool returns:

```
{
  "success": true,
  "output": "",
  "metadata": {}
}
```

16. Browser Service

Purpose

Support Antigravity's embedded browser.

Responsibilities

- Page navigation
 - DOM inspection
 - Screenshot generation
 - Session management
 - Browser automation
-

Frontend Integration

The browser service streams updates directly to:

Browser Workspace Panel

inside Antigravity.

17. Terminal Service

Purpose

Provide persistent terminal execution.

Responsibilities

- Shell execution
 - Process management
 - Output streaming
 - Session persistence
-

Frontend Integration

Terminal output streams directly into:

Terminal Workspace

in Antigravity.

18. Execution Controller

Purpose

Safe execution management.

Responsibilities

- Process isolation
 - Timeout management
 - Retry management
 - Resource limits
 - Sandboxing
-

Execution Flow

```
graph TD;
    Action --> PermissionCheck[Permission Check];
    PermissionCheck --> Sandbox;
    Sandbox --> Execution;
    Execution --> Output;
    Output --> Verification;
```

19. Event Bus

Purpose

Decouple services.

Events

```
TaskCreated
TaskStarted
TaskPaused
TaskCompleted
TaskFailed
```

ToolStarted
ToolFinished
ToolFailed

VerificationPassed
VerificationFailed

Benefits

- Scalability
- Decoupling
- Observability
- Reliability

20. Reporting Service

Purpose

Generate final deliverables.

Outputs

- Investigation reports
- Patch summaries
- Verification reports
- Root cause reports
- Task summaries

Example

Issue Found
↓
Patch Generated
↓
Tests Passed
↓
Verification Passed
↓
Report Generated

21. WebSocket Service

Purpose

Real-time communication with Antigravity.

Streams

Task Stream
Terminal Stream
Browser Stream
Verification Stream
Agent Stream

Benefits

Provides live updates without refreshing.

22. Storage Architecture

Relational Storage

Stores:

- users
- tasks
- plans
- metadata

Recommended:

PostgreSQL

Vector Storage

Stores:

- embeddings
- memories
- repository chunks

Recommended:

Qdrant
or
Weaviate

Object Storage

Stores:

- reports
- artifacts
- screenshots
- logs

Recommended:

S3 Compatible Storage

Cache Layer

Stores:

- session data
- temporary state
- task cache

Recommended:

Redis

23. Observability Architecture

Logging

Capture:

- API calls
- Tool usage
- Agent decisions
- Errors

Metrics

Track:

- Task duration
- Tool success rate
- Verification pass rate
- Memory retrieval latency

Tracing

Every task receives:

Trace ID

allowing complete reconstruction of execution history.

24. Reliability Architecture

Recovery Strategy

The system must recover from:

- crashes
- network failures
- model outages
- tool failures

Recovery Process

Failure
↓
Checkpoint
↓
Restore State
↓
Continue

Checkpoint System

Store:

- current task
- active plan
- memory state
- execution state

This enables long-horizon resilience.

25. Scalability Architecture

Horizontal Scaling

Services should scale independently.

Examples:

```
Planner Pods
Reasoner Pods
Verifier Pods
```

Queue-Based Execution

Recommended:

```
Task Queue
↓
Worker Pool
↓
Execution
```

Benefits:

- Parallelism
 - Reliability
 - Cost control
-

26. Antigravity IDE Optimization

Backend must support:

Browser Workspace

Real-time browser state.

Terminal Workspace

Persistent terminal sessions.

Source Code Workspace

Repository synchronization.

Chat Workspace

Streaming reasoning summaries.

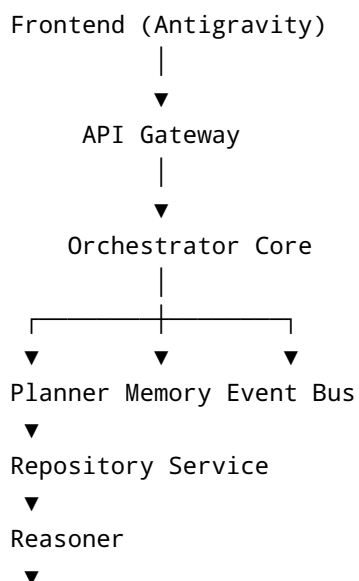
Execution Timeline

Live event stream.

Memory Workspace

Instant memory retrieval.

27. Production Architecture Summary



```
Verifier
▼
Tool Layer
▼
Execution Controller
▼
Storage Layer
```

28. Final Backend Definition

The Agent Phantom Recovery backend is a distributed execution platform built around:

```
Orchestration
+
Planning
+
Reasoning
+
Verification
+
Memory
+
Tools
+
Execution
```

Its purpose is not to generate answers.

Its purpose is to investigate, execute, verify, and complete technical work while providing full visibility through the Antigravity IDE workspace.